

Frontier — The New Engineering Disciplines



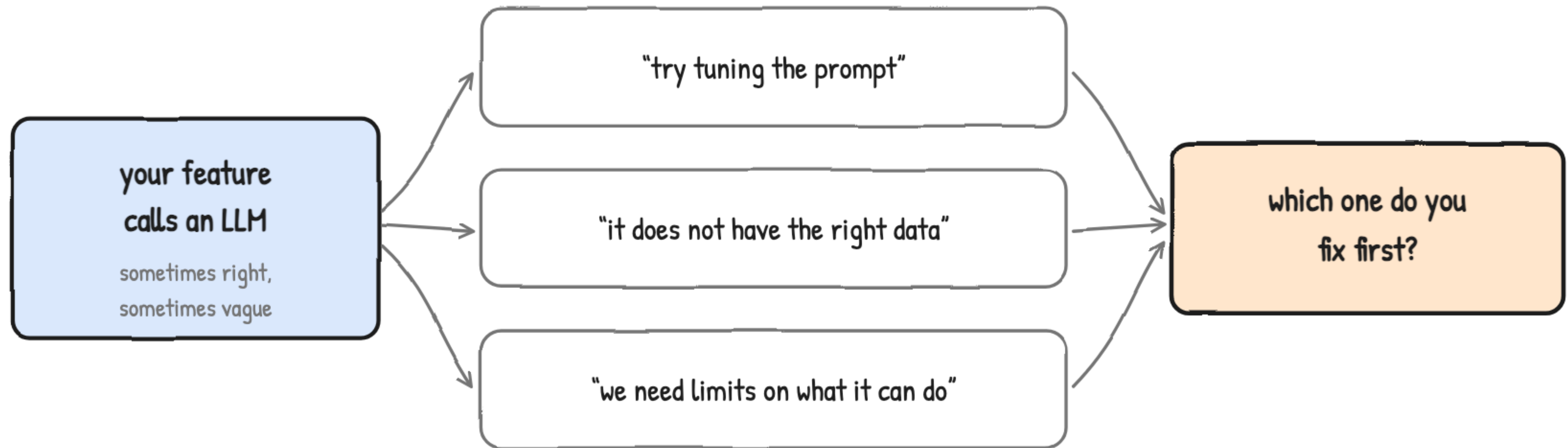
Five practices, and how to know which one you need.

Gourav Shah · School of DevOps & AI · Lesson + Lab + Quiz

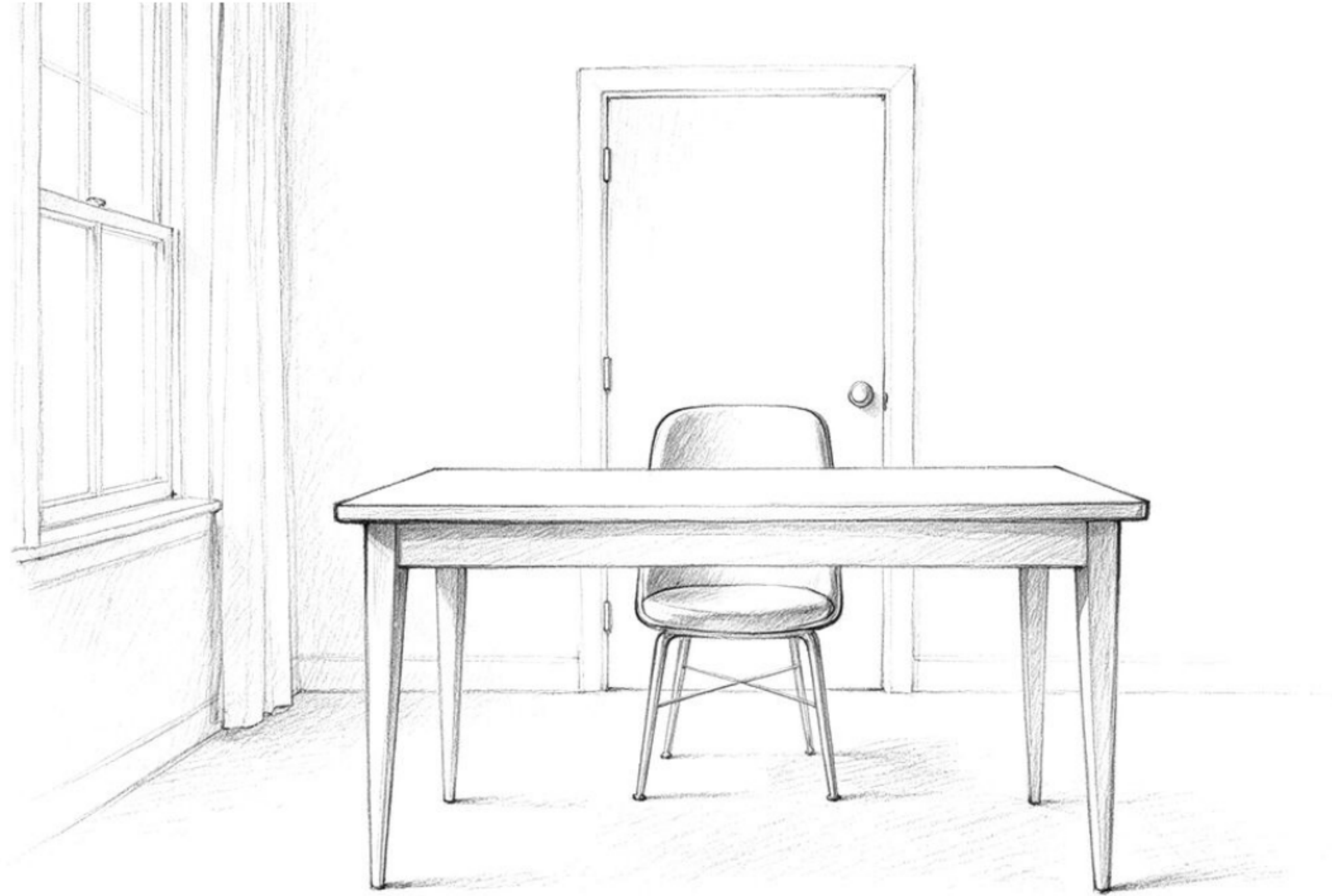


Why this module exists

Three people, three answers, no way to tell who is right



Without a name for each problem, you fix the wrong thing.



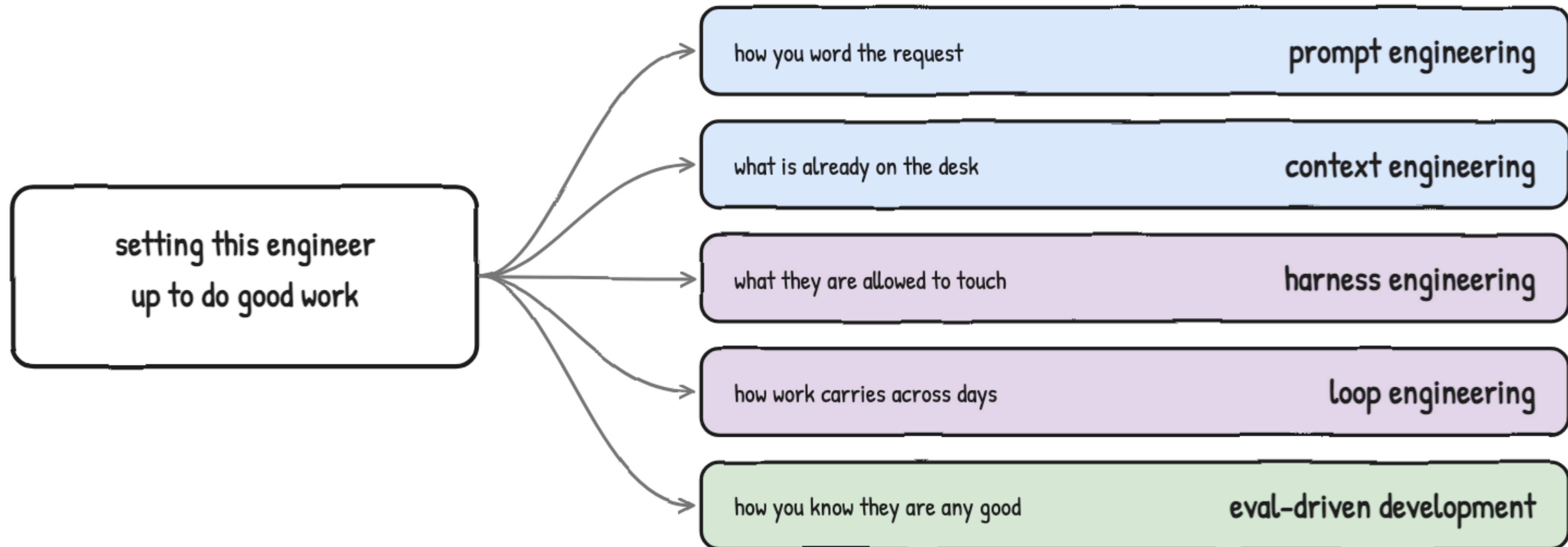
**The analogy: onboarding a capable new engineer with
no memory of your systems**

A capable engineer on day one, with nothing on the desk



Strong skills, but no memory of how your systems work.

Five questions you answer for any new hire

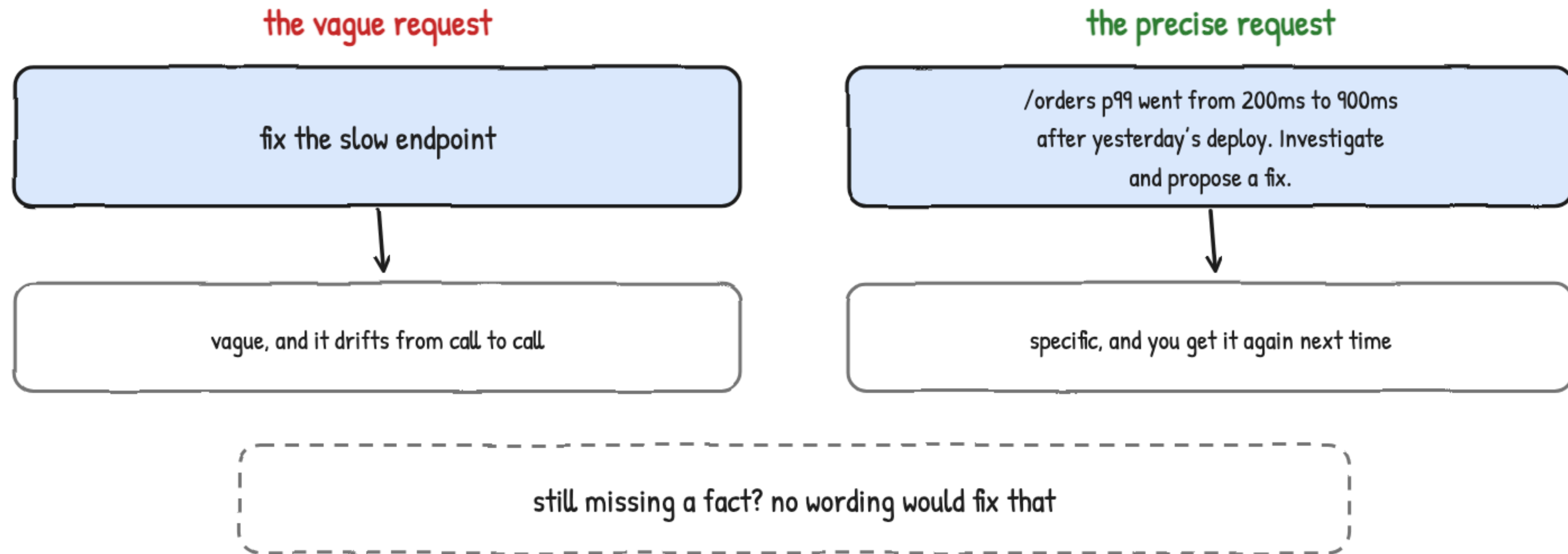


Each one of them has a name now.



Prompt engineering — how you word the single call

Same engineer, same skill, and only the wording changed

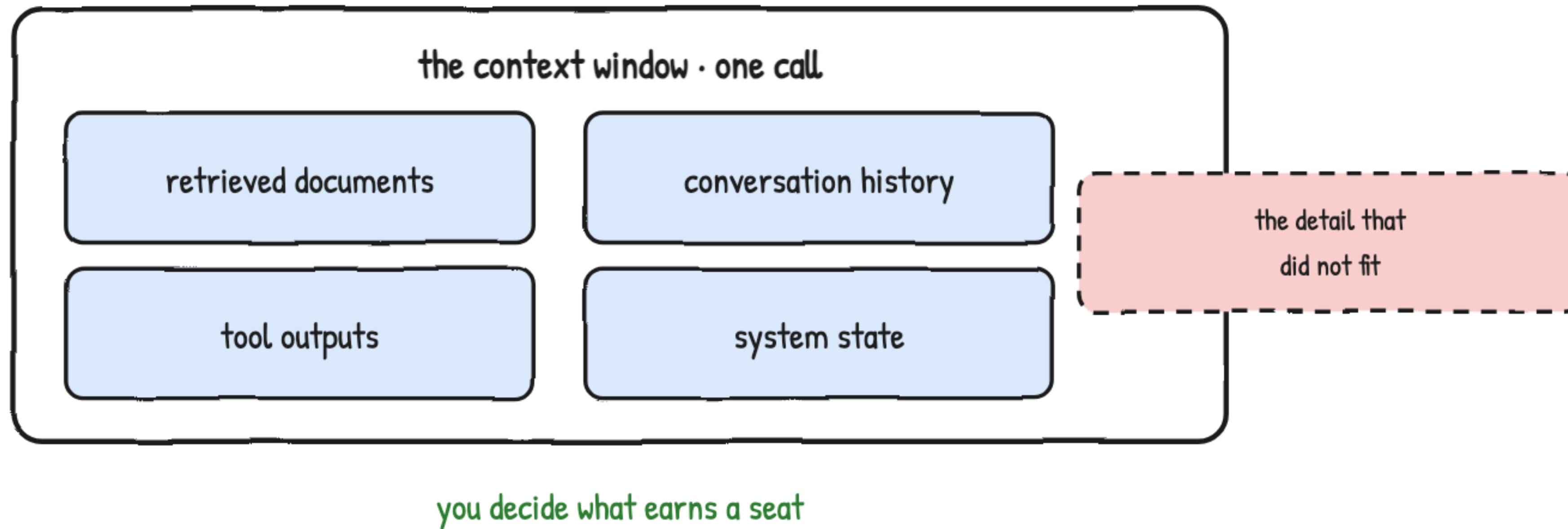


No wording would give the model a fact it never had.



Context engineering – what the model sees, not how you phrase it

One window, and four things want a seat in it

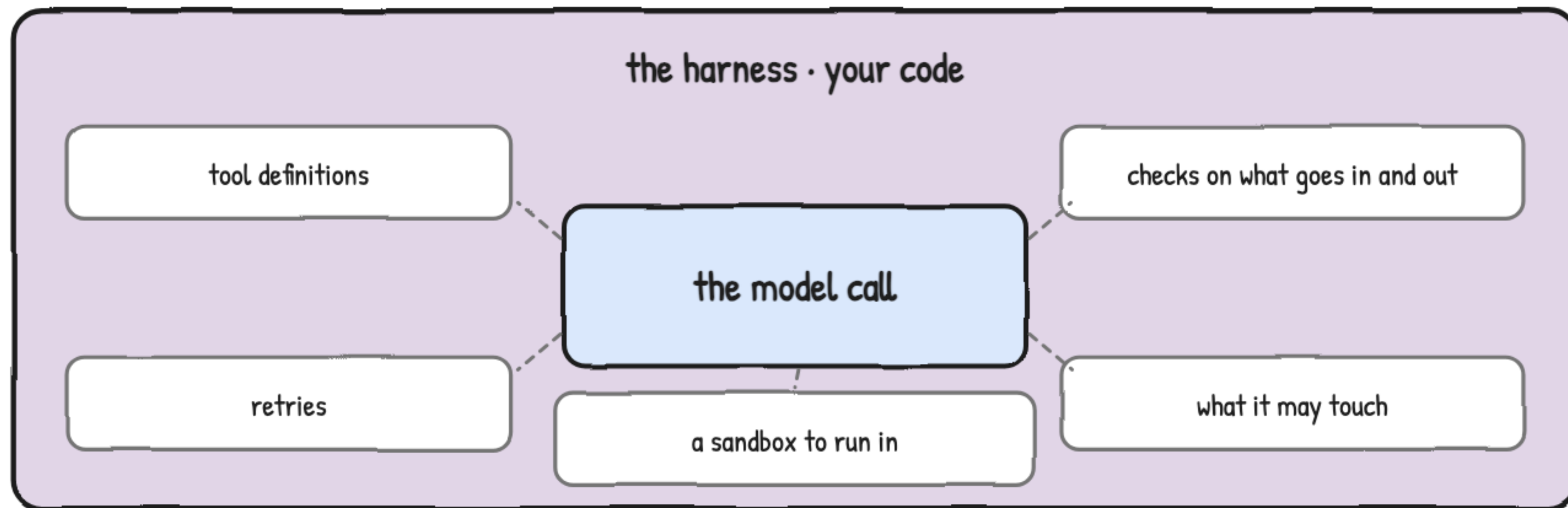


Whatever is not in this window, the model never sees.



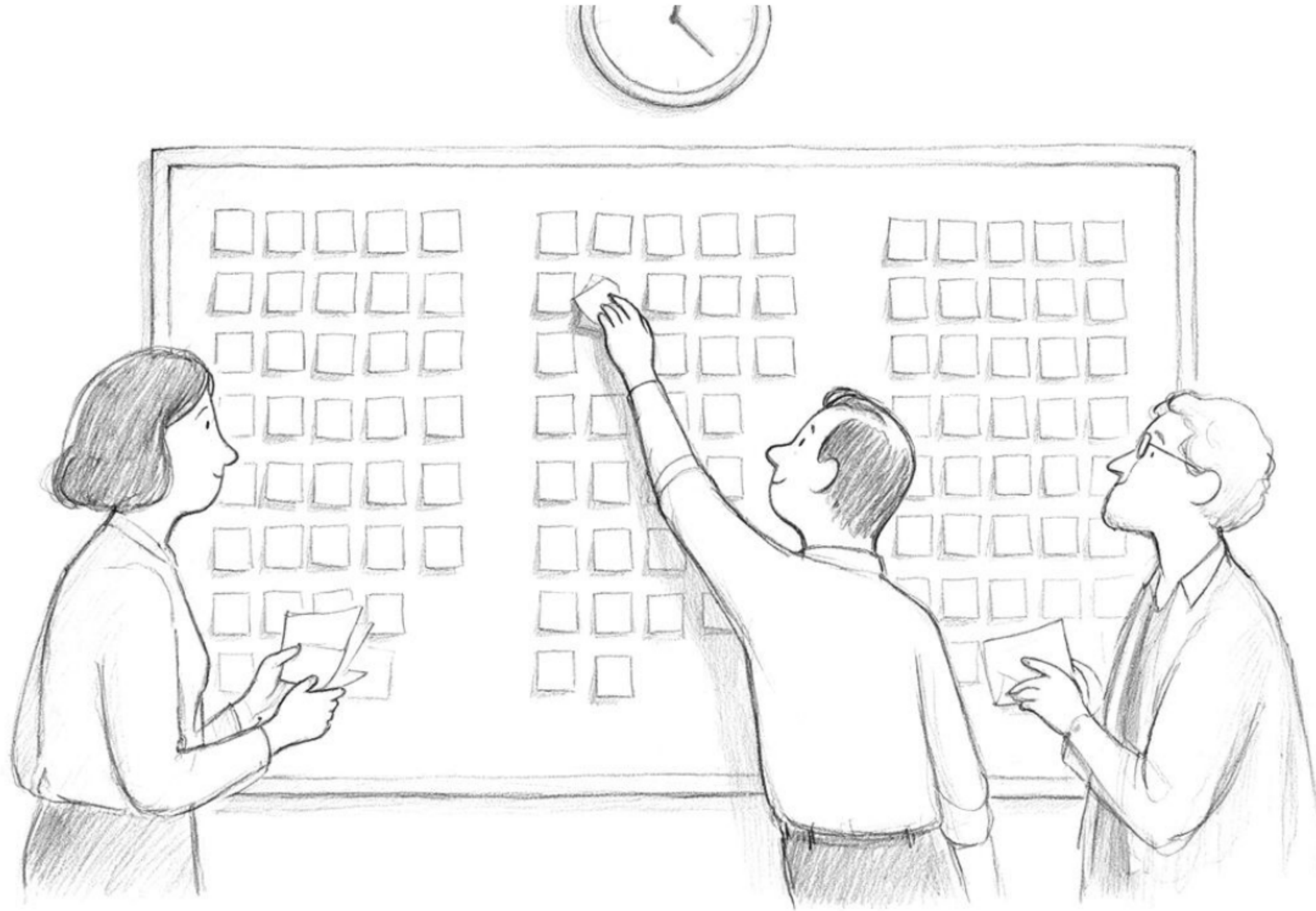
**Harness engineering – the software wrapped around
the call**

The code around the call decides what can go wrong



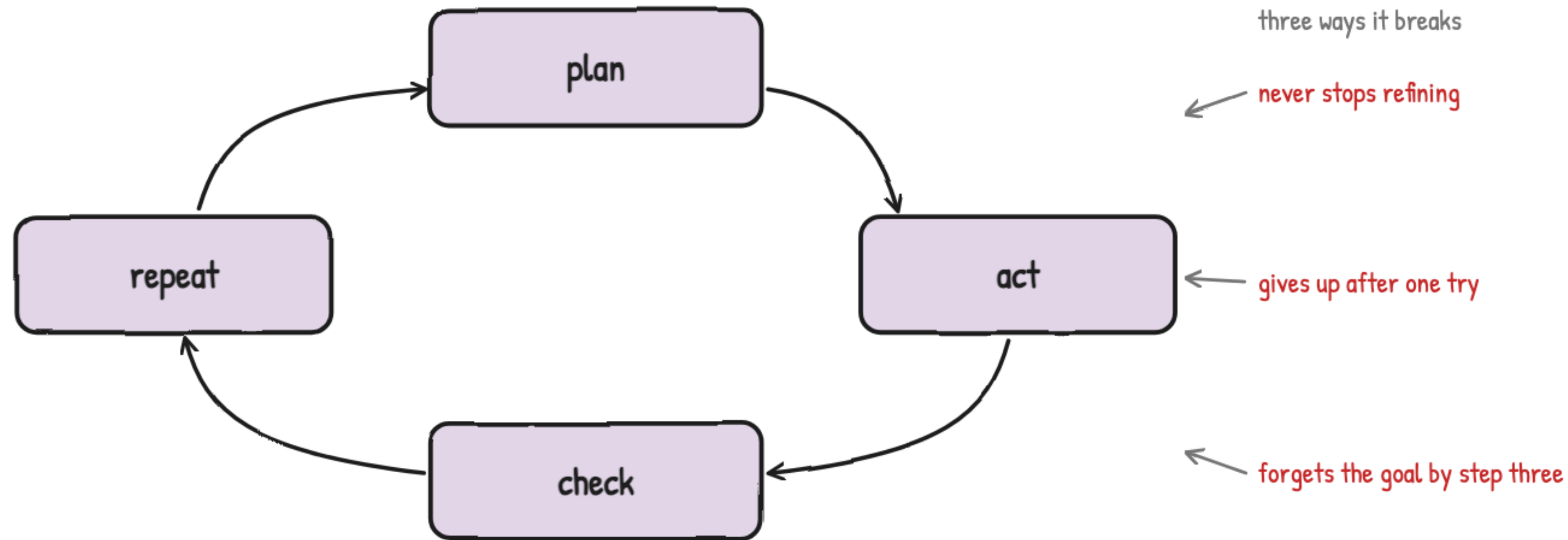
nothing stopped it from deleting a real record

This fix belongs in your code, not in the prompt.



Loop engineering – orchestrating work across multiple steps

Every single call is fine, and still the loop breaks

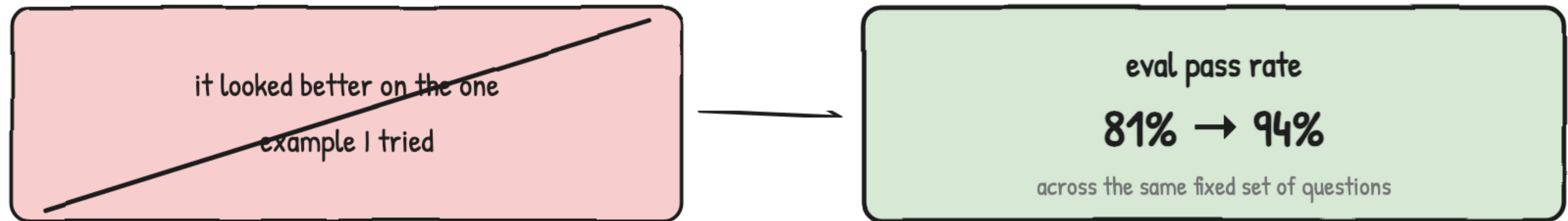


You would redesign the steps, not the wording.



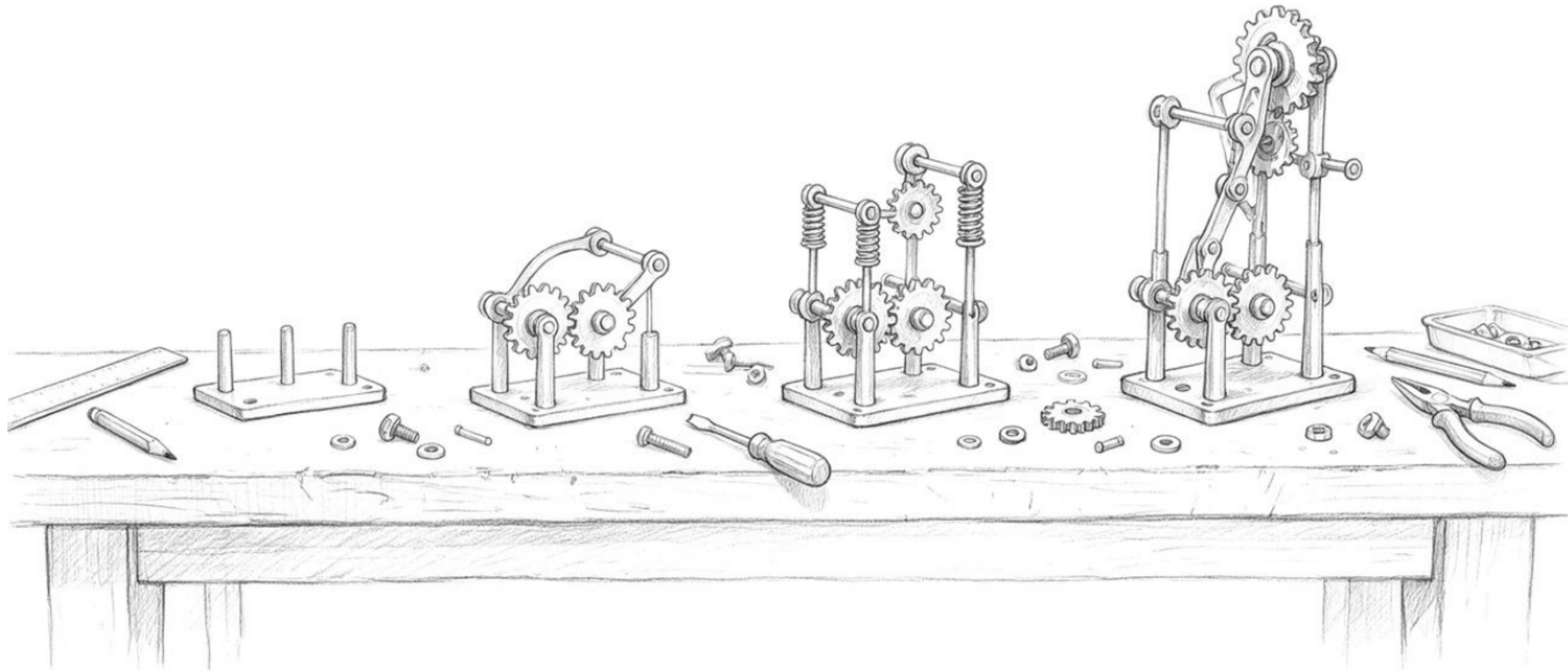
**Eval-driven development – how you know a change
actually helped**

A pass rate you can point at beats a good feeling



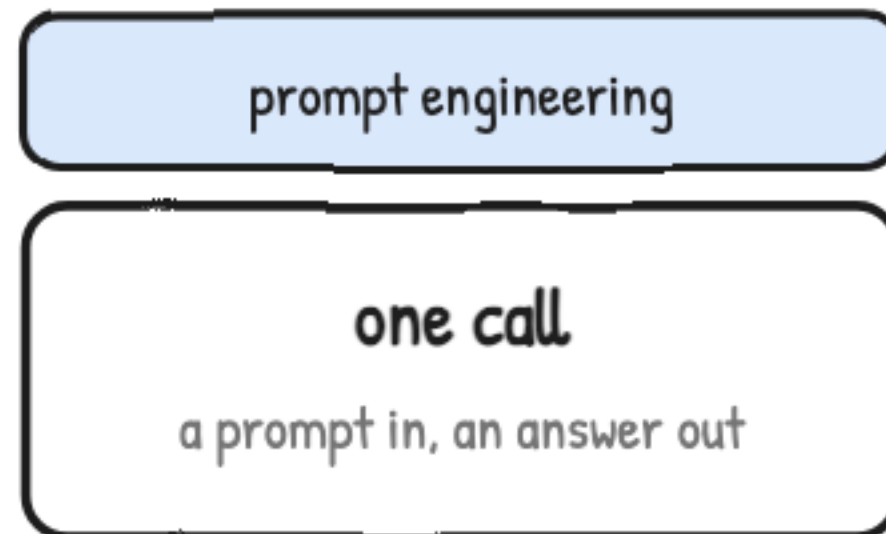
evals are to LLM systems what test suites are to code

One good example is not evidence, and a scorecard is.



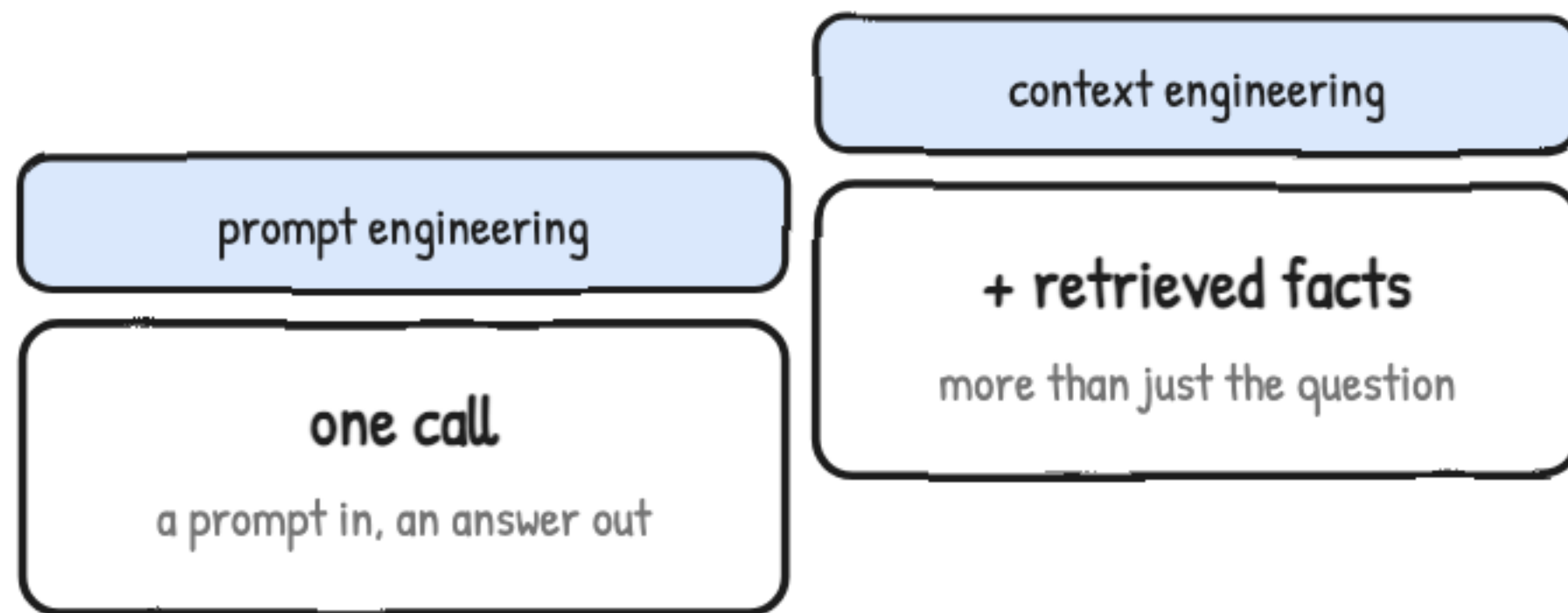
Where these five sit in the build lifecycle

Each layer you add brings its own discipline



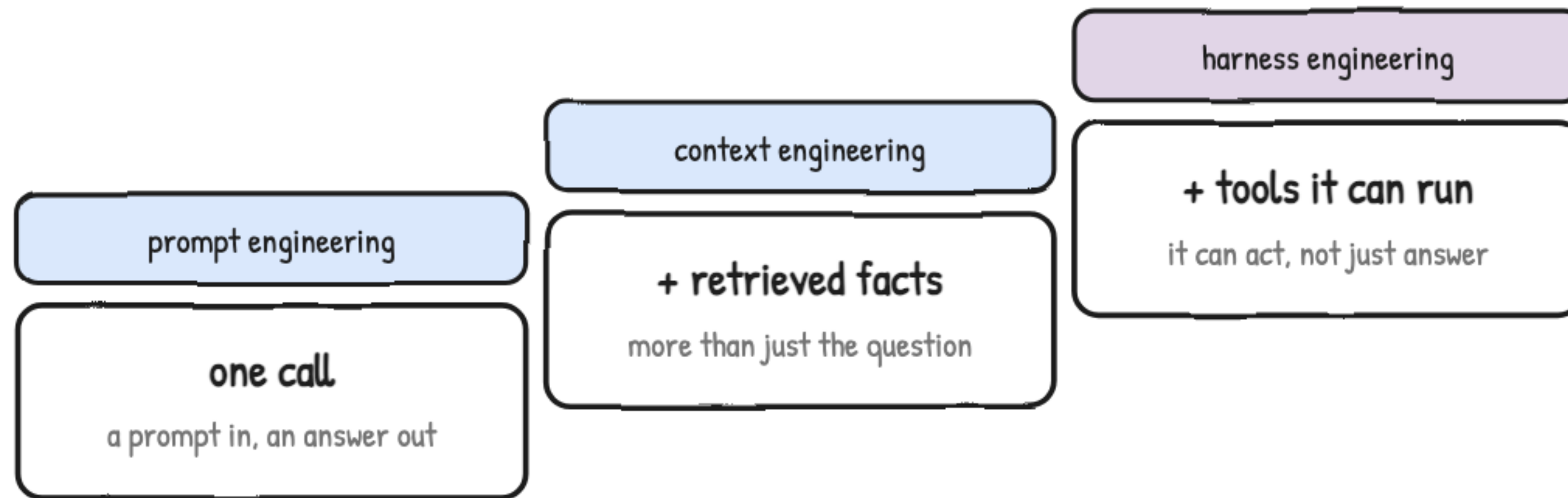
Read it left to right. Evals sit under all four.

Each layer you add brings its own discipline



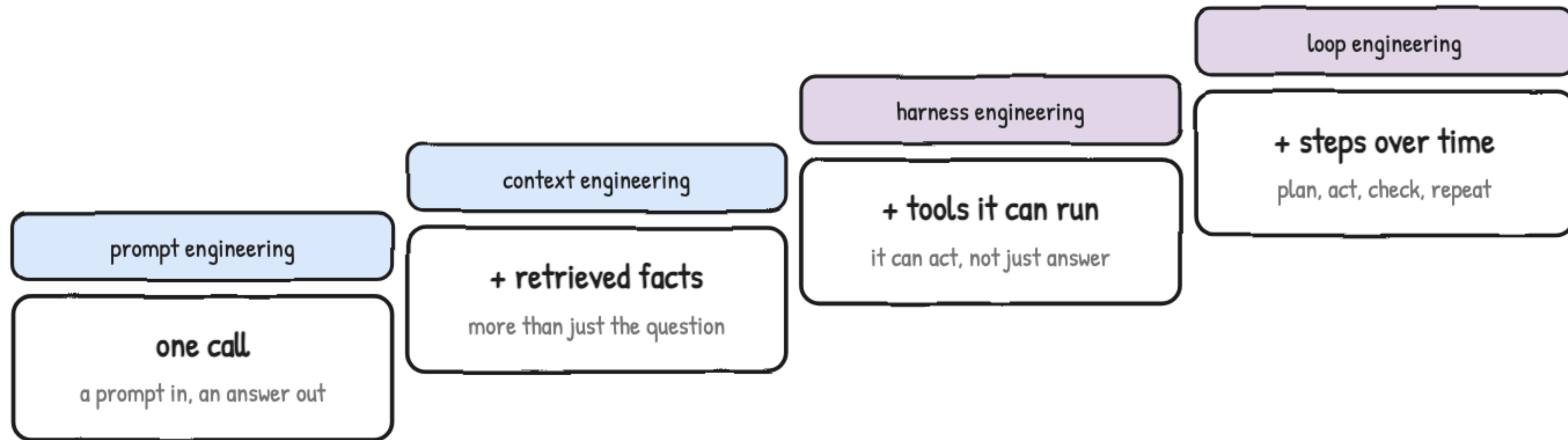
Read it left to right. Evals sit under all four.

Each layer you add brings its own discipline



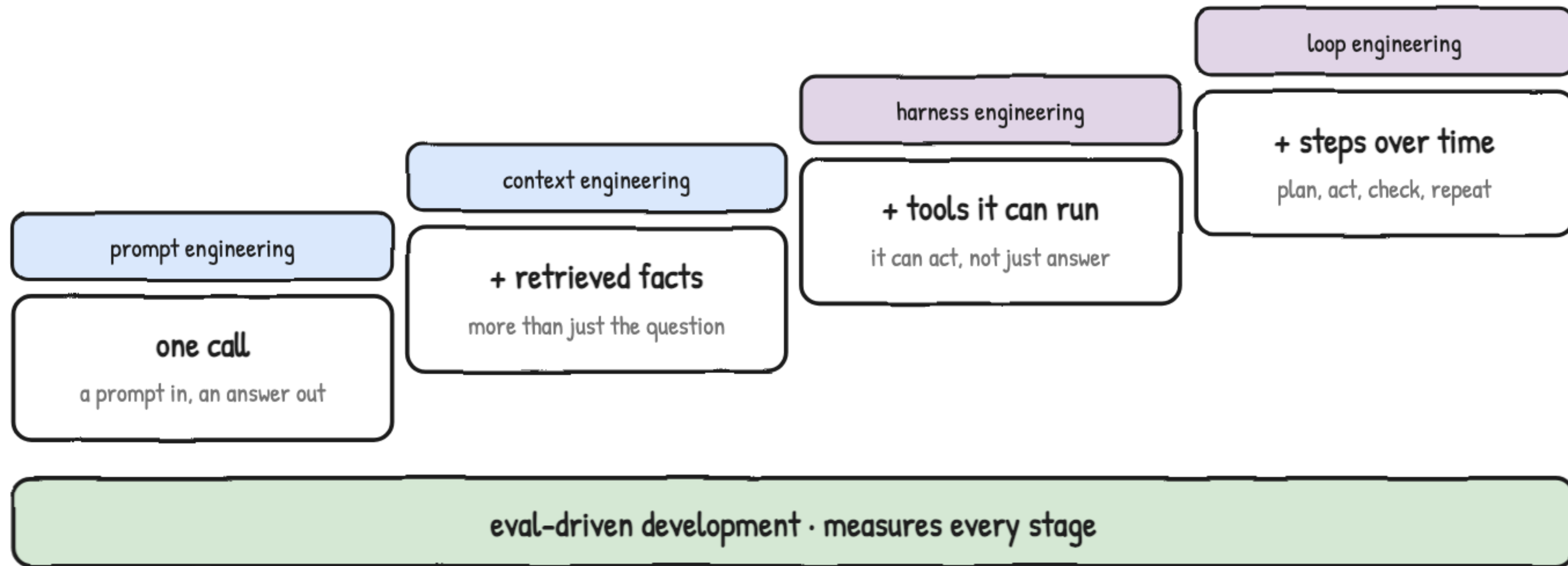
Read it left to right. Evals sit under all four.

Each layer you add brings its own discipline



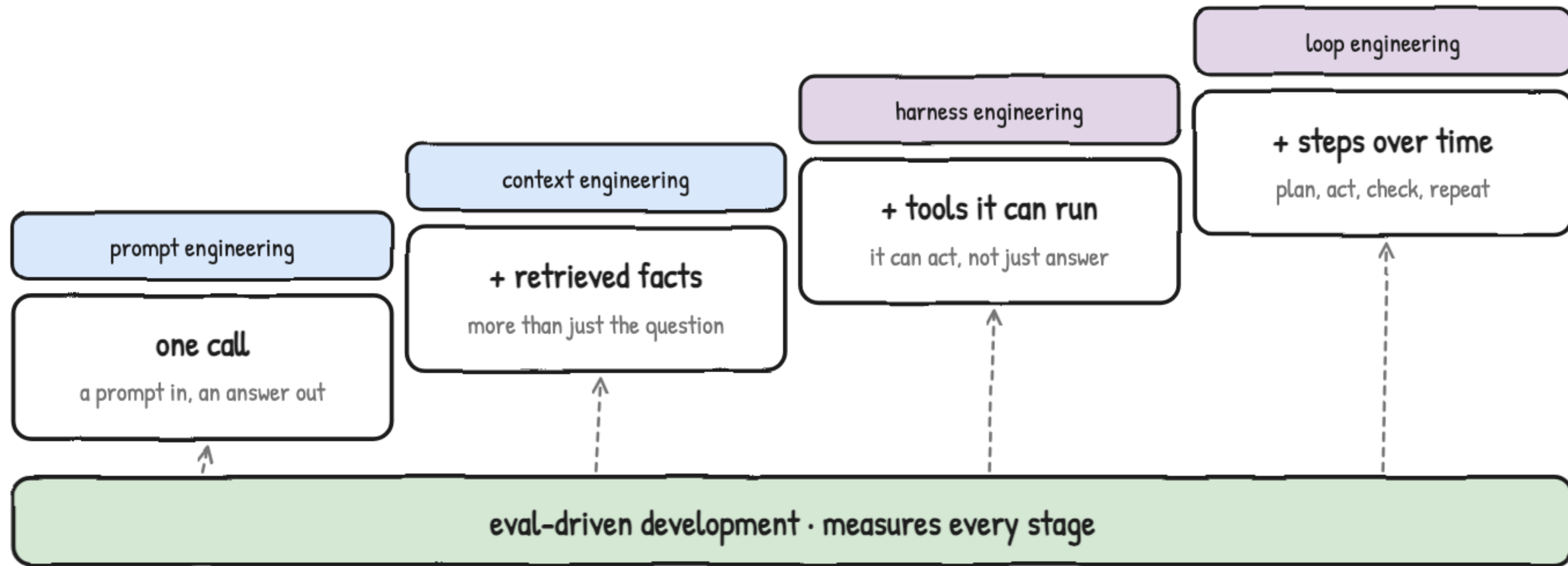
Read it left to right. Evals sit under all four.

Each layer you add brings its own discipline

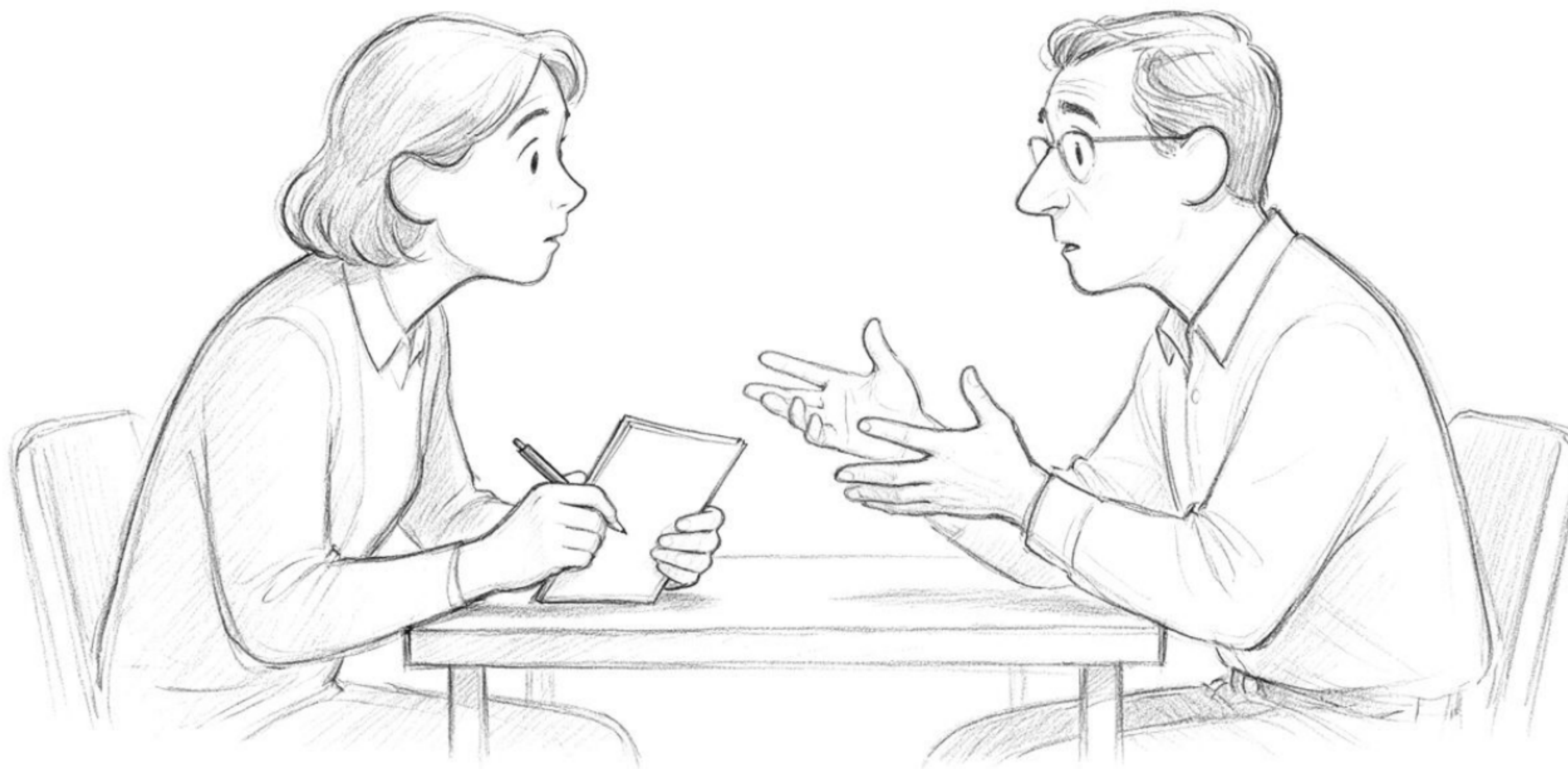


Read it left to right. Evals sit under all four.

Each layer you add brings its own discipline

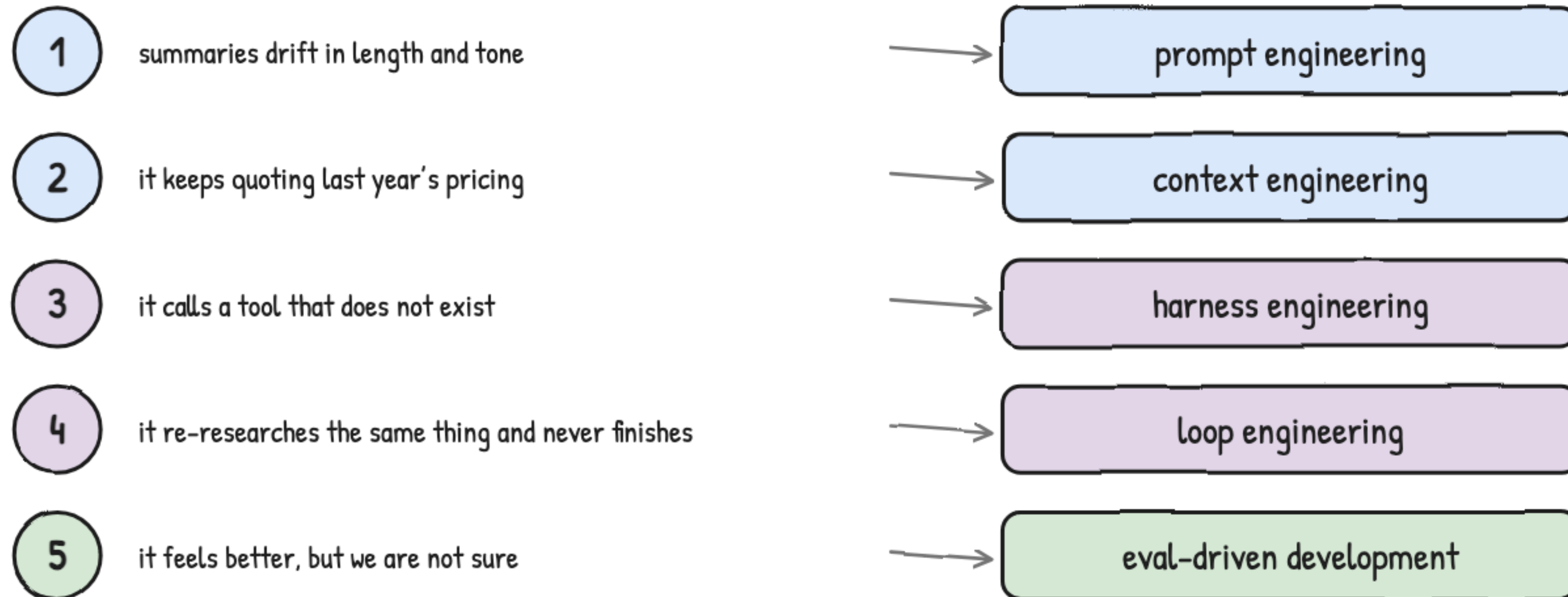


Read it left to right. Evals sit under all four.



Matching a real problem to a discipline

Read the symptom, then name the discipline



Five you have already heard at work.

MODULE 12 TAKEAWAYS

You can name the problem before you fix it



- 1 prompt · how you word the call
- 2 context · what the model sees
- 3 harness · the code around the call
- 4 loop · how many steps, when to stop
- 5 evals · how you know it got better

Now open Module 12 · Lab. · Gourav Shah · School of DevOps & AI